

```
In [2]: import plotly.figure_factory as ff
import geopandas as gpd
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression
from matplotlib.ticker import FuncFormatter
```

Loading Datasets for GeoPanads, loan, and poverty information

```
In [3]: # Load the loan dataset
df_loan = pd.read_csv(
    "https://raw.githubusercontent.com/wujenny214/Policy-Study-into-Grant-Assistance-for-Post-Hurricane-Rebuilding-
```

```
)

df_loan = df_loan[df_loan["GIS State"] == "LA"]
df_loan = df_loan[df_loan["ARS File (Yes/No)"] == "N"]
df_loan = df_loan[df_loan["Current Damage Assessment"] != 9999999.0]
```

/var/folders/xm/yhmrwng57qq9wlcywlm89pxh0000gn/T/ipykernel_98792/4198342093.py:2: DtypeWarning: Columns (13,15) have mixed types. Specify dtype option on import or set low_memory=False.

```
df_loan = pd.read_csv(
```

```
In [4]: # Load SAIPE 2004 Louisiana county-level poverty data
```

```
colspeccs = [
    (0, 2), # 1-2 FIPS State
    (3, 6), # 4-6 FIPS County
    # (7, 15), # 8-15 Est all ages in poverty
    # (16, 24), # 17-24 LCI
    # (25, 33), # 26-33 UCI
    (34, 38), # 35-38 % all ages est
    # (39, 43), # 40-43 % all ages LCI
    # (44, 48), # 45-48 % all ages UCI
    # (49, 57), # 50-57 Est age 0-17
    # (58, 66), # 59-66 LCI
    # (67, 75), # 68-75 UCI
    # (76, 80), # 77-80 % 0-17 est
    # (81, 85), # 82-85 % 0-17 LCI
    # (86, 90), # 87-90 % 0-17 UCI
    # (91, 99), # 92-99 Est related children 5-17 in families
    # (100, 108), # 101-108 LCI
    # (109, 117), # 110-117 UCI
    # (118, 122), # 119-122 % related 5-17 est
    # (123, 127), # 124-127 % related 5-17 LCI
    # (128, 132), # 129-132 % related 5-17 UCI
    # (133, 139), # 134-139 Median HH income
    # (140, 146), # 141-146 Median HH income LCI
    # (147, 153), # 148-153 Median HH income UCI
    # (154, 161), # Estimate of people under age 5 in poverty
    # (162, 169), # 90% confidence interval lower bound of estimate of people under age 5 in poverty
    # (170, 177), # 90% confidence interval upper bound of estimate of people under age 5 in poverty
    # (178, 182), # Estimated percent of people under age 5 in poverty
    # (183, 187), # 90% confidence interval lower bound of estimate of percent of people under age 5 in poverty
    # (189, 192), # 90% confidence interval upper bound of estimate of percent of people under age 5 in poverty
    (193, 238), # State or county name
    # (239, 241), # Two-letter Postal State abbreviation
    # (242, 266), # A tag indicating the file name and date of creation
]
```

```
names = [
    "state_fips",
    "county_fips",
    # "all_ages_pov_est",
    # "all_ages_pov_l",
    # "all_ages_pov_u",
    "all_ages_pov_pct_est",
    # "all_ages_pov_pct_l",
    # "all_ages_pov_pct_u",
    # "age0_17_pov_est",
    # "age0_17_pov_l",
    # "age0_17_pov_u",
    # "age0_17_pov_pct_est",
    # "age0_17_pov_pct_l",
    # "age0_17_pov_pct_u",
    # "rel5_17_pov_est",
    # "rel5_17_pov_l",
    # "rel5_17_pov_u",
    # "rel5_17_pov_pct_est",
    # "rel5_17_pov_pct_l",
    # "rel5_17_pov_pct_u"
```

```

# "rel5_17_pov_u",
# "rel5_17_pov_pct_est",
# "rel5_17_pov_pct_l",
# "rel5_17_pov_pct_u",
# "med_hh_income_est",
# "med_hh_income_l",
# "med_hh_income_u",
# "est_ppl_under5pov",
# "lci_ppl_under5pov",
# "uci_ppl_under5pov",
# "pct_ppl_under5pov_est",
# "pct_ppl_under5pov_l",
# "pct_ppl_under5pov_u",
"parish",
# "state",
# "file_tag",
]

# Read the fixed-width file
url = "https://www2.census.gov/programs-surveys/saipe/datasets/2004/2004-state-and-county/est04-la.txt"
df_poverty = pd.read_fwf(url, colspecs=colspecs, names=names, dtype=str)

```

df_poverty

```

Out [4]:

```

	state_fips	county_fips	all_ages_pov_pct_est	parish
0	22	0	19.2	Louisiana
1	22	1	21.0	Acadia Parish
2	22	3	22.0	Allen Parish
3	22	5	12.0	Ascension Parish
4	22	7	19.3	Assumption Parish
...	...	...	...	...
60	22	119	19.0	Webster Parish
61	22	121	16.8	West Baton Rouge Parish
62	22	123	22.7	West Carroll Parish
63	22	125	20.4	West Feliciana Parish
64	22	127	22.8	Winn Parish

65 rows x 4 columns

```

In [5]:
# # Construct the FIPS codes directly and save them into the dataframe
# df_poverty["FIPS"] = df_poverty.apply(
#     lambda row: row["state_fips"]
#     + ("00" if len(row["county_fips"]) == 1 else "0")
#     + row["county_fips"],
#     axis=1,
# )

# (ALT method) Initialize an empty list to store the FIPS codes
fips_codes = []

# Iterate through each row in the dataframe
for i, row in df_poverty.iterrows():
    # Check the length of the county_fips and construct the FIPS code accordingly
    if len(row["county_fips"]) == 1:
        fips_code = row["state_fips"] + "00" + row["county_fips"]
    elif len(row["county_fips"]) == 2:
        fips_code = row["state_fips"] + "0" + row["county_fips"]
    else:
        fips_code = row["state_fips"] + row["county_fips"]
    fips_codes.append(fips_code)

# Add the constructed FIPS codes as a new column in the dataframe
df_poverty["FIPS"] = fips_codes

df_poverty = df_poverty[["FIPS", "parish", "all_ages_pov_pct_est"]].copy()
df_poverty = df_poverty[1:]
df_poverty

```

```
Out [5]:
```

	FIPS	parish	all_ages_pov_pct_est
1	22001	Acadia Parish	21.0
2	22003	Allen Parish	22.0
3	22005	Ascension Parish	12.0
4	22007	Assumption Parish	19.3
5	22009	Avoyelles Parish	24.8
...	...	...	...
60	22119	Webster Parish	19.0
61	22121	West Baton Rouge Parish	16.8
62	22123	West Carroll Parish	22.7
63	22125	West Feliciana Parish	20.4
64	22127	Winn Parish	22.8

64 rows x 3 columns

Plotting Louisiana

```
In [6]: # Download all US counties (2023, 1:5m cartographic boundaries) from Census
url = "https://www2.census.gov/geo/tiger/GENZ2023/shp/cb_2023_us_county_5m.zip"
counties = gpd.read_file(url)

# Keep only Louisiana (STATEFP = '22')
la = counties[counties["STATEFP"] == "22"].copy()

# Keep useful fields; GEOID is the full 5-digit county/parish FIPS
la = la[["GEOID", "STATEFP", "COUNTYFP", "NAME", "geometry"]]

# Save as GeoJSON (Leaflet/Mapbox/Folium/Plotly-ready)
la.to_file("louisiana_counties_fips.geojson", driver="GeoJSON")

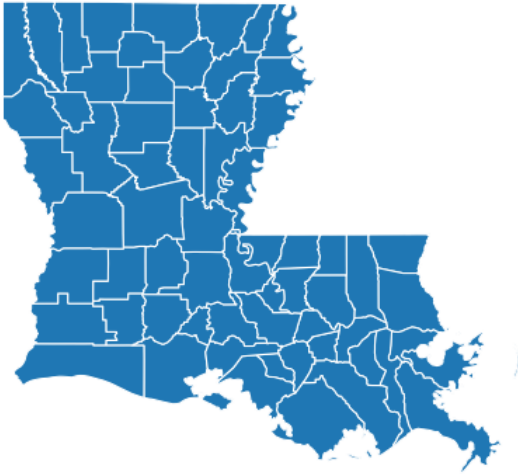
# Mapping Louisiana parishes
ax = la.plot(edgecolor="white", linewidth=0.8)
ax.set_axis_off()

la
```

```
Out [6]:
```

	GEOID	STATEFP	COUNTYFP	NAME	geometry
103	22061	22	061	Lincoln	POLYGON ((-92.88093 32.58526, -92.86661 32.607...
392	22095	22	095	St. John the Baptist	POLYGON ((-90.68727 30.0372, -90.66997 30.0476...
409	22029	22	029	Concordia	POLYGON ((-91.8633 31.35445, -91.84502 31.3759...
570	22003	22	003	Allen	POLYGON ((-93.13119 30.51828, -93.13081 30.564...
628	22093	22	093	St. James	POLYGON ((-90.96374 30.06683, -90.93559 30.085...
...	...	...	...	...	...
3095	22083	22	083	Richland	POLYGON ((-92.03117 32.2814, -92.01982 32.2976...
3118	22037	22	037	East Feliciana	POLYGON ((-91.32071 30.69942, -91.2931 30.7155...
3126	22051	22	051	Jefferson	POLYGON ((-90.28006 29.99171, -90.27986 29.998...
3153	22049	22	049	Jackson	POLYGON ((-92.81715 32.1617, -92.79882 32.1813...
3160	22115	22	115	Vernon	POLYGON ((-93.56976 30.99672, -93.56798 31.001...

64 rows x 5 columns



```
In [7]: # Concatenate the la dataframe with the df_poverty dataframe to include the geometry variable
df_poverty_with_geometry = pd.concat(
    [df_poverty.set_index("FIPS"), la.set_index("GEOID")["geometry"]],
    axis=1,
    join="inner",
).reset_index()

df_poverty_with_geometry
```

```
Out [7]:
```

	index	parish	all_ages_pov_pct_est	geometry
0	22001	Acadia Parish	21.0	POLYGON ((-92.63216 30.31429, -92.6195 30.3400...
1	22003	Allen Parish	22.0	POLYGON ((-93.13119 30.51828, -93.13081 30.564...
2	22005	Ascension Parish	12.0	POLYGON ((-91.10656 30.10852, -91.08983 30.108...
3	22007	Assumption Parish	19.3	POLYGON ((-91.25894 30.00067, -91.23635 30.000...
4	22009	Avoyelles Parish	24.8	POLYGON ((-92.28074 30.96507, -92.2086 30.9621...
...	...	...	...	...
59	22119	Webster Parish	19.0	POLYGON ((-93.51632 33.00254, -93.52099 33.018...
60	22121	West Baton Rouge Parish	16.8	POLYGON ((-91.48173 30.51103, -91.41482 30.511...
61	22123	West Carroll Parish	22.7	POLYGON ((-91.63888 32.70247, -91.61772 32.720...
62	22125	West Feliciana Parish	20.4	MULTIPOLYGON (((-91.66251 30.97036, -91.64877 ...
63	22127	Winn Parish	22.8	POLYGON ((-92.97604 31.71242, -92.9686 31.7310...

64 rows x 4 columns

```
In [8]: import pandas as pd
import matplotlib.pyplot as plt

# Create bins manually
bins = [0, 5, 10, 15, 20, 25, 100]
labels = ["0-5%", "6-10%", "11-15%", "16-20%", "21-25%", "25%+"]

# Ensure the column is numeric before applying pd.cut
df_poverty_with_geometry["all_ages_pov_pct_est"] = pd.to_numeric(
    df_poverty_with_geometry["all_ages_pov_pct_est"], errors="coerce"
)

df_poverty_with_geometry["pov_bin"] = pd.cut(
    df_poverty_with_geometry["all_ages_pov_pct_est"],
    bins=bins,
    labels=labels,
    include_lowest=True,
)

df_poverty_with_geometry = gpd.GeoDataFrame(
    df_poverty_with_geometry, geometry="geometry", crs="EPSG:4269"
)

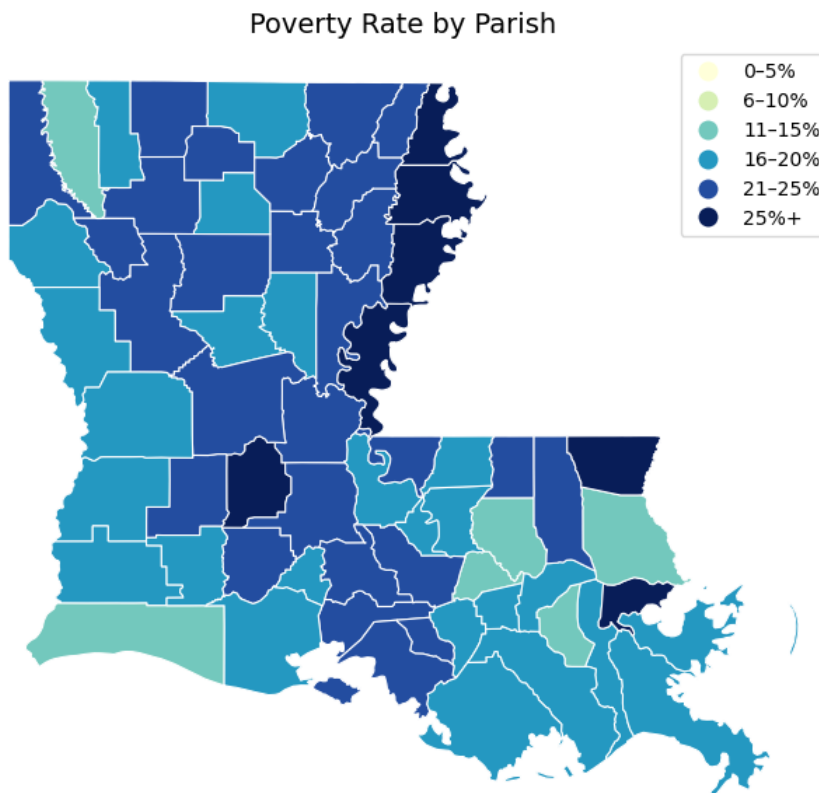
df_poverty_with_geometry
```

```
Out [8]:
```

	index	parish	all_ages_pov_pct_est	geometry	pov_bin
0	22001	Acadia Parish	21.0	POLYGON ((-92.63216 30.31429, -92.6195 30.3400...	21-25%
1	22003	Allen Parish	22.0	POLYGON ((-93.13119 30.51828, -93.13081 30.564...	21-25%
2	22005	Ascension Parish	12.0	POLYGON ((-91.10656 30.10852, -91.08983 30.108...	11-15%
3	22007	Assumption Parish	19.3	POLYGON ((-91.25894 30.00067, -91.23635 30.000...	16-20%
4	22009	Avoyelles Parish	24.8	POLYGON ((-92.28074 30.96507, -92.2086 30.9621...	21-25%
...	...	...	...	...	...
59	22119	Webster Parish	19.0	POLYGON ((-93.51632 33.00254, -93.52099 33.018...	16-20%
60	22121	West Baton Rouge Parish	16.8	POLYGON ((-91.48173 30.51103, -91.41482 30.511...	16-20%
61	22123	West Carroll Parish	22.7	POLYGON ((-91.63888 32.70247, -91.61772 32.720...	21-25%
62	22125	West Feliciana Parish	20.4	MULTIPOLYGON (((-91.66251 30.97036, -91.64877 ...	21-25%
63	22127	Winn Parish	22.8	POLYGON ((-92.97604 31.71242, -92.9686 31.7310...	21-25%

64 rows x 5 columns

```
In [9]: # Plot with grey for missing
fig, ax = plt.subplots(figsize=(8, 8))
df_poverty_with_geometry.plot(
    column="pov_bin",
    cmap="YlGnBu",
    linewidth=0.8,
    edgecolor="white",
    legend=True,
    ax=ax,
    missing_kwds={
        "color": "lightgrey",
        "edgecolor": "white",
        "hatch": "///",
        "label": "No data",
    },
)
ax.set_title("Poverty Rate by Parish", fontsize=14)
ax.axis("off")
plt.show()
```



```

In [10]: # To get accurate figures for the net grant amounts, the difference between "current" (Current Total DOB Amount) a
# be subtracted from the sum of the compensation and additional compensation grants (TOTAL_CLOSING_AMOUNT).

# Difference between "current" (Current Total DOB Amount) and "closing" (Closing Total DOB Amount) DOB
df_loan["delta"] = (
    df_loan["Current Total DOB Amount (no Legal Fees removed)"]
    - df_loan["Closing Total DOB Amount"]
)

# Calculate gross average loan amount
df_loan["gross_loan_amount"] = df_loan["TOTAL_CLOSING_AMOUNT"] - df_loan["delta"]

df_loan.head()

```

Out[10]:

	Unnamed: 0	Structure Type	GIS City	GIS State	GIS Zip	PARISH	Closing Option	TOTAL_CLOSING_AMOUNT	Total CG Amount	Total ACG Amunt	...	PSV at Closing
0	9613.0	Single (including mobile home)	METAIRIE	LA	70001	Jefferson	1.0	150000.00	95213.27	39036.73	...	172000.0
1	21349.0	Duplex (with one owner-occupied unit)	METAIRIE	LA	70001	Jefferson	2.0	150000.00	150000.00	0.00	...	250000.0
2	25325.0	Duplex (with one owner-occupied unit)	METAIRIE	LA	70001	Jefferson	1.0	3450.14	3450.14	0.00	...	240000.0
3	27236.0	Duplex (with one owner-occupied unit)	METAIRIE	LA	70001	Jefferson	1.0	10665.59	10665.59	0.00	...	136000.0
4	36177.0	Single (including mobile home)	METAIRIE	LA	70001	Jefferson	1.0	131947.98	101947.98	0.00	...	345000.0

5 rows x 39 columns

```

In [11]: from matplotlib.ticker import FuncFormatter

# Filter the dataframe for Low Income rows
low_income_df = df_loan[df_loan["Total ACG Amunt"] > 0]

# Filter the dataframe for High Income rows
high_income_df = df_loan[df_loan["Total ACG Amunt"] == 0]

# Create subplots
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))

# Define formatter for dollar amounts with commas
formatter = FuncFormatter(lambda x, _: f"${x:,.0f}")

# Plot Low Income dots
ax1.scatter(
    low_income_df["Current PSV"],
    low_income_df["Current Damage Assessment"],
    c="red",
    alpha=0.4,
    s=100
)

```

Loading [MathJax]/jax/output/CommonHTML/fonts/TeX/fontdata.js

```

)
ax1.plot(
    [low_income_df["Current PSV"].min(), low_income_df["Current PSV"].max()],
    [low_income_df["Current PSV"].min(), low_income_df["Current PSV"].max()],
    color="black",
    linestyle="--",
)
ax1.set_xlabel("Pre-Storm Value ($)")
ax1.set_ylabel("Damage Assessment ($)")
ax1.set_title("Low Property Value Households", fontsize=12)
ax1.xaxis.set_major_formatter(formatter)
ax1.yaxis.set_major_formatter(formatter)

# Plot High Income
ax2.scatter(
    high_income_df["Current PSV"],
    high_income_df["Current Damage Assessment"],
    c="blue",
    alpha=0.4,
    s=10,
)
ax2.plot(
    [high_income_df["Current PSV"].min(), high_income_df["Current PSV"].max()],
    [high_income_df["Current PSV"].min(), high_income_df["Current PSV"].max()],
    color="black",
    linestyle="--",
)
ax2.set_xlabel("Pre-Storm Value ($)")
ax2.set_title("High Property Value Households", fontsize=12)
ax2.xaxis.set_major_formatter(formatter)
ax2.yaxis.set_major_formatter(formatter)

# Remove gridlines, legends, and extra text
ax1.grid(False)
ax2.grid(False)
ax1.legend().remove()
ax2.legend().remove()

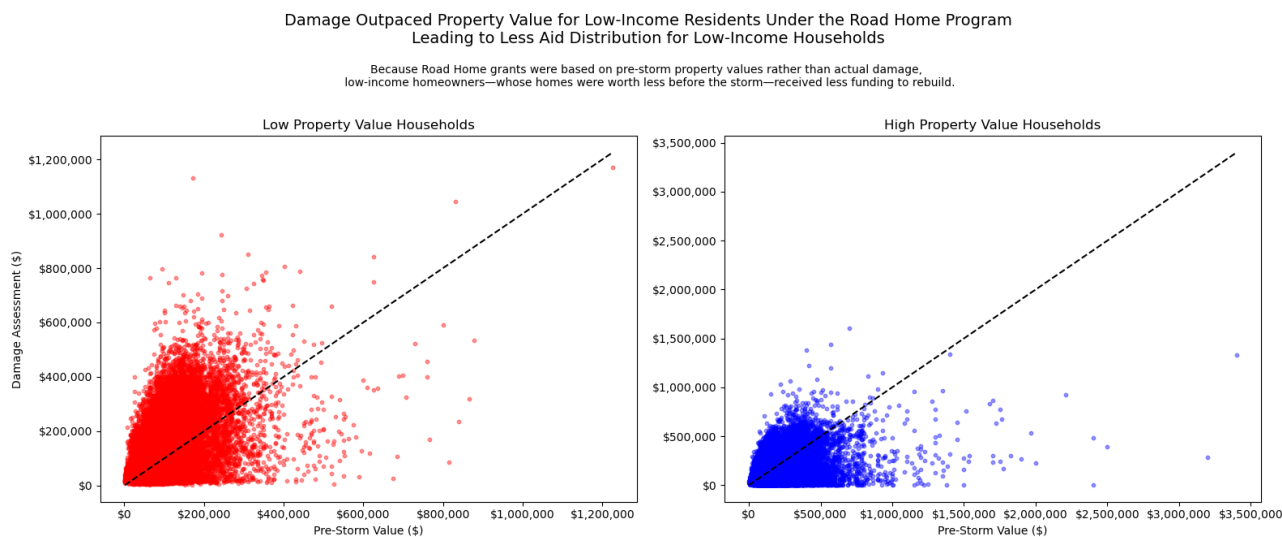
fig.suptitle(
    "Damage Outpaced Property Value for Low-Income Residents Under the Road Home Program\nLeading to Less Aid Distr
    fontsize=14,
    y=1.1,
)
fig.text(
    0.5,
    0.95,
    "Because Road Home grants were based on pre-storm property values rather than actual damage, \n low-income home
    ha="center",
    fontsize=10,
)

# Simplify layout
plt.tight_layout()
plt.show()

```

No artists with labels found to put in legend. Note that artists whose label start with an underscore are ignored when legend() is called with no argument.

No artists with labels found to put in legend. Note that artists whose label start with an underscore are ignored when legend() is called with no argument.



```
In [12]: import matplotlib as mpl

mpl.rcParams.update(
    {
        "svg.fonttype": "none", # keep text editable in PPT
        "svg.image_inline": True, # embed raster parts
    }
)

import matplotlib.pyplot as plt
from matplotlib.ticker import FuncFormatter

# Filter the dataframe for Low Income rows
low_income_df = df_loan[df_loan["Total ACG Amunt"] > 0]
high_income_df = df_loan[df_loan["Total ACG Amunt"] == 0]

# plot
fig, ax = plt.subplots(figsize=(10, 8), dpi=300) # dpi controls rasterized quality
formatter = FuncFormatter(lambda x, _: f"${x:,.0f}")

# scatter layers rasterized so PPT can import
ax.scatter(
    high_income_df["Current PSV"],
    high_income_df["Current Damage Assessment"],
    c="red",
    alpha=0.4,
    s=10,
    label="High Property Value Households",
    rasterized=True,
)
ax.scatter(
    low_income_df["Current PSV"],
    low_income_df["Current Damage Assessment"],
    c="lightblue",
    edgecolors="white",
    alpha=0.3,
    s=10,
    label="Low Property Value Households",
    rasterized=True,
)

# diagonal reference line
all_psv = df_loan["Current PSV"]
ax.plot(
    [all_psv.min(), all_psv.max()],
    [all_psv.min(), all_psv.max()],
    color="black",
    linestyle="--",
    linewidth=1,
    label="Equal Value Line",
)

# limits, labels, formatting
max_damage = max(
    low_income_df["Current Damage Assessment"].max(),
    high_income_df["Current Damage Assessment"].max(),
)
```



```

)
max_psv = df_loan["Current PSV"].max()
ax.set_xlim(0, max_psv)
ax.set_ylim(0, max_damage)
ax.set_xlabel("Pre-Storm Value ($)", fontsize=11)
ax.set_ylabel("Damage Assessment ($)", fontsize=11)
ax.xaxis.set_major_formatter(formatter)
ax.yaxis.set_major_formatter(formatter)
ax.grid(False)

# titles
fig.suptitle(
    "Damage Outpaced Property Value for Low-Income Residents Under the Road Home Program\n"
    "Leading to Less Aid Distribution for Low-Income Households",
    fontsize=14,
    y=1.02,
)
fig.text(
    0.5,
    0.92,
    "Because Road Home grants were based on pre-storm property values rather than actual damage,\n"
    "low-income homeowners—whose homes were worth less before the storm—received less funding to rebuild.",
    ha="center",
    fontsize=10,
)

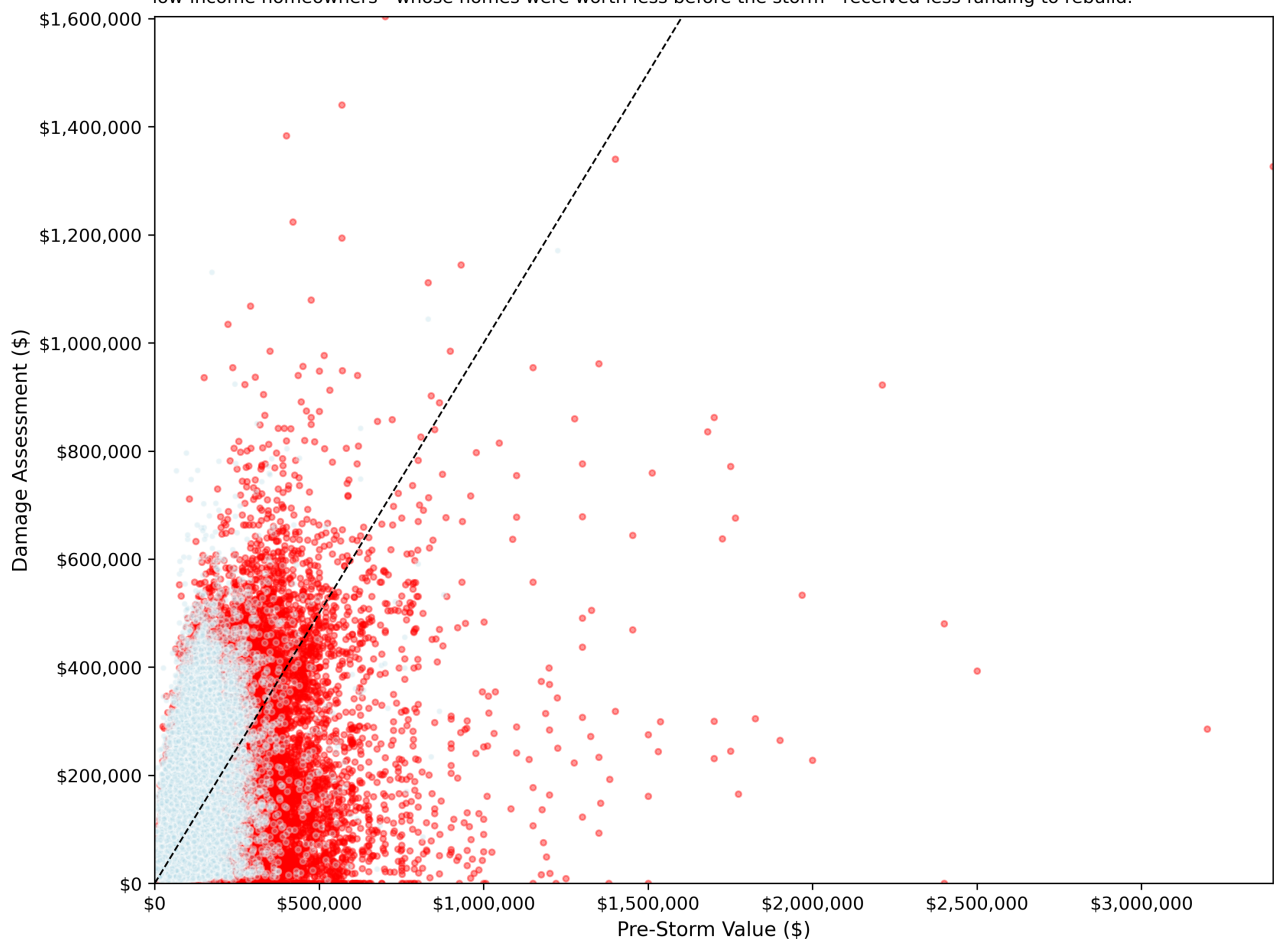
plt.tight_layout()

# save PPT-safe SVG (points are rasterized inside the SVG)
raw_svg = "jw_memo_plot01.svg"
fig.savefig(raw_svg, format="svg", transparent=True)

```

Damage Outpaced Property Value for Low-Income Residents Under the Road Home Program Leading to Less Aid Distribution for Low-Income Households

Because Road Home grants were based on pre-storm property values rather than actual damage,
low-income homeowners—whose homes were worth less before the storm—received less funding to rebuild.



```

In [13]: for i in df_loan.index:
          if df_loan.at[i, "Current Damage Assessment"] > df_loan.at[i, "Current PSV"]:
              df_loan.at[i, "page_psv"] = 1

```

Loading [MathJax/jax/output/CommonHTML/fonts/TeX/fontdata.js]

```

    else:
        df_loan.at[i, "damage_psv"] = 0

df_loan.head()

```

Out[13]:

	Unnamed: 0	Structure Type	GIS City	GIS State	GIS Zip	PARISH	Closing Option	TOTAL_CLOSING_AMOUNT	Total CG Amount	Total ACG Amunt	...	Closer file - PSN Option With No ACG - excluding Sold Home Inelig Deceaser and file: closer with RHE & IMM only -23,91%
0	9613.0	Single (including mobile home)	METAIRIE	LA	70001	Jefferson	1.0	150000.00	95213.27	39036.73	...	↑
1	21349.0	Duplex (with one owner-occupied unit)	METAIRIE	LA	70001	Jefferson	2.0	150000.00	150000.00	0.00	...	↑
2	25325.0	Duplex (with one owner-occupied unit)	METAIRIE	LA	70001	Jefferson	1.0	3450.14	3450.14	0.00	...	↑
3	27236.0	Duplex (with one owner-occupied unit)	METAIRIE	LA	70001	Jefferson	1.0	10665.59	10665.59	0.00	...	↑
4	36177.0	Single (including mobile home)	METAIRIE	LA	70001	Jefferson	1.0	131947.98	101947.98	0.00	...	↑

5 rows x 40 columns

```

In [45]: # Filter the dataframe for Low Income rows
low_income_df = df_loan[df_loan["Total ACG Amunt"] > 0]
high_income_df = df_loan[df_loan["Total ACG Amunt"] == 0]

# Calculate proportions
low_income_proportion = low_income_df["damage_psv"].mean()
high_income_proportion = high_income_df["damage_psv"].mean()

# Create bar plot with flipped axes
proportions = [low_income_proportion, high_income_proportion]
labels = ["Low Income", "High Income"]

plt.figure(figsize=(8, 6))
bars = sns.barplot(y=labels, x=proportions, orient="h")

# Set x-axis limits to 0-100%
plt.xlim(0, 1.0)

# Format x-axis as percentage
plt.gca().xaxis.set_major_formatter(plt.FuncFormatter(lambda x, _: "{:.0%}".format(x)))

# Add percentage labels to the right of bars
for i, (bar, proportion) in enumerate(zip(bars.patches, proportions)):
    bars.text(
        proportion / 2,
        bar.get_y() + bar.get_height() / 2.0,
        f"{proportion:.1%}",
        ha="center",
        va="center",
        fontsize=14,
    )

```

```
        fontweight="bold",
        color="white",
    )

    plt.xlabel("Proportion of Homes with Damage Greater Than Pre-Storm Value of Home")
    plt.ylabel("")
    plt.title(
        "Low Income Households More Adversely Impacted by Road Home's\nProperty-Value Based Funding Calculations."
    )
    plt.rcParams.update(
        {
            "svg.fonttype": "none", # keep text editable in PPT
        }
    )
    plt.savefig("jw_memo_plot02.svg", format="svg", transparent=True)
    bars.spines["top"].set_visible(False)
    bars.spines["right"].set_visible(False)
    plt.show()
```

